ADR-001 — Arquitectura segura con servicios gestionados (FDSI-GP-05)

Oct 13, 2026 · Grupo 5 FDSI

Estado y contexto

Estado: Propuesto (pendiente de revisión del equipo GP-05).

El asistente académico con RAG del repo hoy no tiene ningún freno al consumo: la versión Unsecure deja que cualquiera pregunte sin login, sin cuota y trayendo hasta 50 fragmentos por consulta. Ese es el riesgo OWASP LLM10:2025 *Unbounded Consumption*: además de tumbar el servicio, un atacante puede disparar la factura del LLM (*denial of wallet*).

La versión Secure actual mete los controles como clases dentro de una sola API FastAPI con Redis local (rate limiter por tokens, caché semántica en memoria, tope `MAX_CHUNKS_RAG = 5`). Eso sirve para medir en una laptop, pero tiene tres huecos:

- Todo control vive en el mismo proceso que atacan: si la API se satura, los controles caen con ella.
- El `usuario_id` llega en el body, así que un atacante puede rotarlo y saltarse la cuota.
- No hay perímetro ni capa de red: no se filtra tráfico volumétrico antes de que llegue a Python.

En clase se pidió llevar esto a una arquitectura con servicios gestionados, por capas (defensa en profundidad), con un AI Gateway delante de los modelos, un orquestador de agente, RAG sobre documentos `.md`, varios proveedores de LLM y herramientas expuestas por MCP. Este ADR registra esa decisión.

Decisión

Desplegamos el asistente en Google Cloud con tres zonas separadas (pública, perímetro, privada) y un AI Gateway gestionado (Apigee) como único punto por donde entra una pregunta y por donde sale cualquier llamada a un LLM. Así el consumo se corta antes de gastar un token, no después.

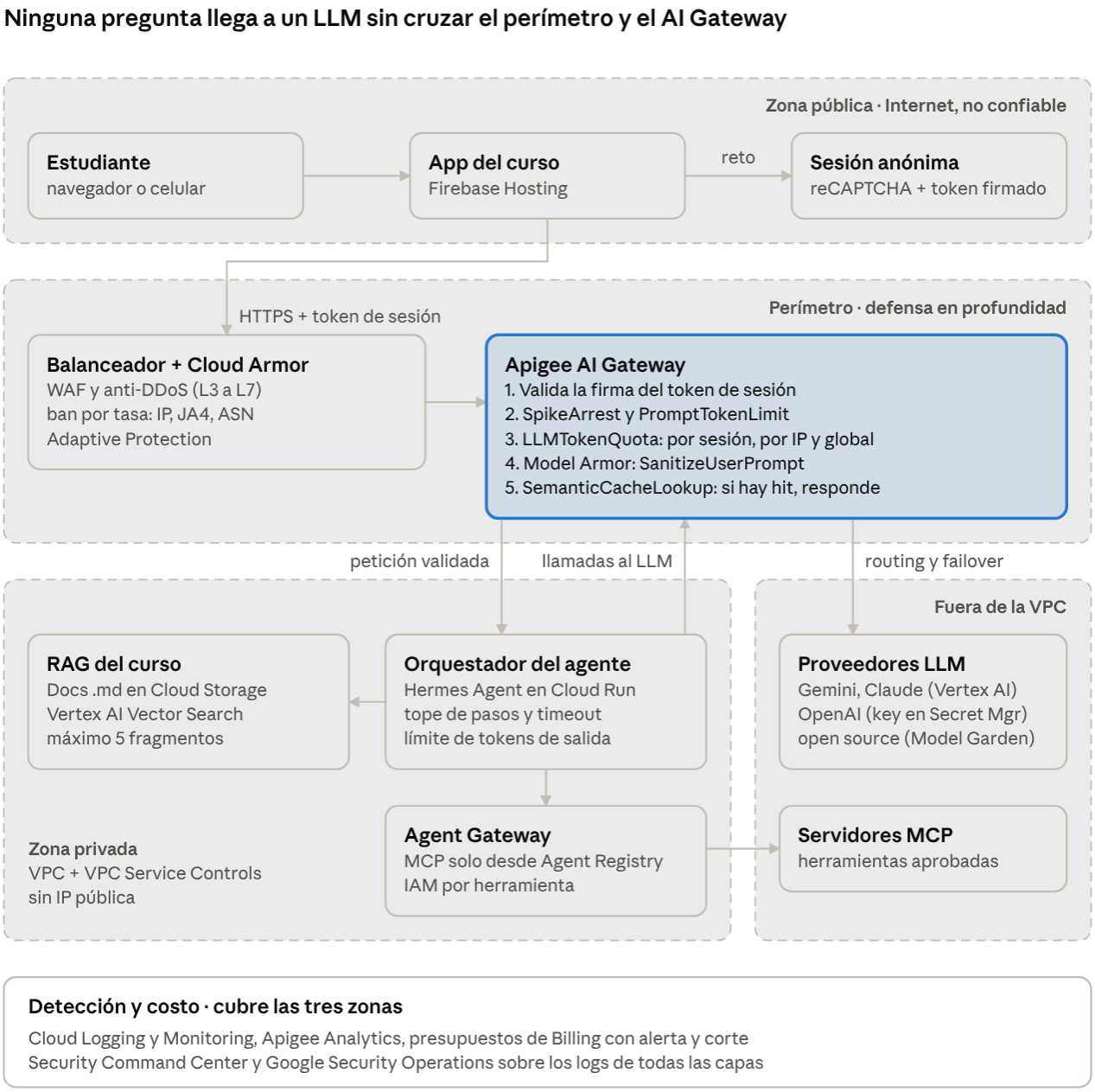
Las reglas que fija esta decisión:

- **Defensa en profundidad:** cada capa frena un tipo de abuso distinto. Si una falla, la siguiente sigue en pie: Cloud Armor frena volumen, la sesión anónima con reCAPTCHA frena los bots y el AI Gateway frena el gasto en tokens.

- **Sin login, la identidad es una sesión anónima firmada.** El chatbot es público: tras pasar reCAPTCHA, la app recibe un token de sesión corto firmado por el backend. El AI Gateway valida esa firma y usa su `sub` como llave de cuota, junto con la IP y la huella TLS; además hay un tope global de tokens por día para todo el servicio. Rotar `usuario_id` deja de servir.
- **Cuota por tokens reales, no por peticiones.** Es la misma idea del Token Bucket del repo, pero aplicada en el gateway con las políticas `PromptTokenLimit` y `LLMTokenQuota`.
- **El orquestador del agente vive en zona privada**, sin IP pública. Solo recibe tráfico del gateway y solo sale a LLMs o herramientas a través de gateways. Eso evita el pivoteo: comprometer el agente no da acceso directo a internet ni a otras cargas.
- **Varios proveedores de LLM detrás del mismo gateway** (Gemini, Claude, OpenAI, open source). La cuota y la caché se aplican igual sin importar el modelo, y si un proveedor falla se cambia de ruta.
- **Herramientas solo por MCP aprobado.** El agente llama tools a través de Agent Gateway y solo las que estén registradas en Agent Registry.

La versión de laboratorio (FastAPI + Redis) se mantiene para medir. Este ADR es la arquitectura objetivo que esa versión simula.

Diagrama



arquitectura segura · 3 zonas + capa de detección

La petición baja de la zona pública al perímetro y solo entra a la zona privada ya validada; el orquestador vuelve a salir por Apigee para hablar con cualquier LLM y por Agent Gateway para usar herramientas.

Componentes por capa

Cada capa tiene un servicio gestionado, un control concreto y el ataque del proyecto que frena. La última columna dice qué parte del repo lo simula hoy.

Capa	Servicio	Control que aplica	Frena	Equivalente en el repo
1. Borde	Cloud Armor + balanceador HTTPS	WAF, anti-DDoS; *throttle* o *rate-based ban* por IP, huella TLS JA4 o ASN	Flooding volumétrico, bots	No existe (pendiente)
2. Identidad	Sesión anónima firmada + reCAPTCHA	Sin login: reto anti-bot y token de sesión corto y firmado; cuota por sesión, IP y huella TLS	Bots, rotación de `usuario_id`	`usuario_id` en el body (débil)
3. AI Gateway (ingreso)	Apigee AI gateway	`VerifyJWT`, `SpikeArrest`, `PromptTokenLimit`, `LLMTokenQuota`	Ráfagas, prompts gigantes, gasto diario sin tope	`RateLimiter` en `secure/rate_limiter.py`
3. AI Gateway (ingreso)	Model Armor vía `SanitizeUserPrompt`	Filtra prompt injection, jailbreak, datos sensibles y URLs maliciosas	Prompts que fuerzan al agente a gastar o a filtrar datos	No existe
3. AI Gateway (ingreso)	Caché semántica de Apigee (`SemanticCacheLookup` / `SemanticCachePopulate`)	Responde preguntas parecidas sin llamar al LLM	Repetición (T03)	`SemanticCache` en memoria
4. Orquestador	Hermes Agent en Cloud Run (ingreso interno)	Tope de pasos por pregunta, timeout, límite de tokens de salida	Bucles del agente, respuestas infinitas	No existe
4. RAG	Cloud Storage (docs `.md`) + Vertex AI Vector Search	Máximo 5 fragmentos por consulta	Contexto masivo (T02)	`MAX_CHUNKS_RAG = 5`
5. Egreso a LLMs	Apigee (mismo gateway, flujo de salida)	Routing entre Gemini, Claude, OpenAI y open source; failover; `SanitizeModelResponse`; cuenta tokens reales	Caída de un proveedor, fuga en respuestas	`ask_llm` en `common/llm_client.py`
5. Egreso a tools	Agent Gateway + Agent Registry	Solo servidores MCP aprobados, IAM por herramienta, Model Armor	Pivoteo desde el agente, tools no autorizadas	No existe
5. Red privada	VPC + VPC Service Controls, Secret Manager	Sin IP pública, datos dentro del perímetro, API keys fuera del código	Exfiltración, keys filtradas	`.env` local
6. Detección y costo	Cloud Logging/Monitoring, presupuestos de Billing, Security Command Center, Google Security Operations	Alertas de consumo, corte automático al pasar el presupuesto, correlación de eventos	Denial of wallet que se escapa de las capas anteriores	Métricas de Locust

Los nombres de política de Apigee y Model Armor salen de la documentación de Google; los valores de cuota (tokens por día, tope de pasos) se calibran con los datos de T01–T05.

Flujo de una pregunta

Una pregunta solo gasta tokens si pasa los pasos 1 a 6; cualquier rechazo antes de eso cuesta cero en el LLM.

1. El estudiante abre el chatbot sin login; reCAPTCHA descarta bots y el backend le entrega un token de sesión anónimo, corto y firmado.

2. La pregunta llega al balanceador. Cloud Armor bloquea IPs o huellas TLS que superen la tasa y aplica el WAF.
3. Apigee valida la firma del token de sesión y toma su `sub` como identidad. `SpikeArrest` corta ráfagas y `PromptTokenLimit` rechaza prompts demasiado largos (HTTP 429/400).
4. `LLMTokenQuota` revisa la cuota de la sesión, la de la IP y el tope global del servicio. Si alguna se agotó, 429 sin tocar el LLM.
5. Model Armor revisa el prompt (`SanitizeUserPrompt`): inyección, jailbreak o datos sensibles se bloquean aquí.
6. `SemanticCacheLookup` busca una pregunta parecida. Si hay hit, responde de caché y termina.
7. Si no, el orquestador (zona privada) recupera máximo 5 fragmentos del RAG y arma el prompt.
8. La llamada al modelo sale otra vez por Apigee: routing al proveedor, failover si falla y `SanitizeModelResponse` sobre la respuesta.
9. Si el agente necesita una herramienta, la pide por MCP a través de Agent Gateway; solo se permiten las registradas.
10. Apigee descuenta los tokens reales de la respuesta, guarda en caché (`SemanticCachePopulate`) y todo queda en logs; Billing corta si se supera el presupuesto.

Alternativas consideradas

Google Cloud con Apigee y Azure con API Management son las dos opciones donde cuota por tokens, caché semántica, filtro de prompts y MCP vienen como políticas del mismo gateway. Elegimos Google porque su borde (Cloud Armor) agrupa por más señales que solo la IP; Azure queda como alternativa equivalente (detalle abajo).

Opción	Cómo se vería	A favor	En contra	Decisión
Google Cloud + Apigee AI gateway	La de este ADR	Políticas nativas para tokens, caché semántica, Model Armor y MCP; Gemini y Claude disponibles en Vertex AI	Apigee es de pago; más piezas que configurar	Elegida

Opción	Cómo se vería	A favor	En contra	Decisión
AWS	CloudFront + AWS WAF/Shield, API Gateway con usage plans, Bedrock + Guardrails, ElastiCache para caché y cuotas	El equipo ya conoce AWS; Bedrock da varios modelos	La cuota por tokens y la caché semántica hay que programarlas en Lambda	Plan B
Gateway open source (LiteLLM o Kong AI Gateway) en contenedor	Mismo diseño por zonas, gateway autoadministrado	Barato, corre en laptop, sirve para la demo	Nosotros operamos el gateway y su disponibilidad; sin anti-DDoS gestionado	Útil para el laboratorio
Quedarse en el monolito actual (FastAPI + Redis)	Controles como clases dentro de la API	Fácil de medir y aislar cada control	El control cae junto con la API atacada; sin perímetro ni identidad fuerte	Solo como entorno de pruebas

Opción Azure, capa por capa

Azure cubre las seis capas con servicios gestionados equivalentes: el AI gateway de Azure API Management (APIM) trae cuota por tokens, caché semántica, filtro de contenido y gobierno de MCP como políticas nativas, igual que Apigee.

Capa	Google (este ADR)	Azure	Control en Azure
1. Borde	Cloud Armor	Azure Front Door + WAF, Azure DDoS Protection	Rate limit por IP de socket en ventanas de 1 o 5 minutos; al pasar el umbral bloquea el resto de la ventana
2. Identidad	Sesión anónima + reCAPTCHA	Reto anti-bot + token de sesión emitido por el backend	Sin login; APIM valida la firma del token de sesión con `validate-jwt`

Capa	Google (este ADR)	Azure	Control en Azure
3. AI Gateway	Apigee	Azure API Management, AI gateway	`llm-token-limit`: límite por minuto o cuota de tokens por llave (suscripción, IP o expresión, por ejemplo el `sub` del JWT); puede precalcular los tokens del prompt y rechazar sin llamar al modelo
3. Filtro de prompts	Model Armor	Azure AI Content Safety vía `llm-content-safety`	Categorías de daño con umbral, `shield-prompt` para ataques al prompt, blocklists propias; responde 403. También revisa tráfico de tools MCP
3. Caché semántica	Política de Apigee	`llm-semantic-cache-lookup` / `llm-semantic-cache-store`	Necesita Azure Managed Redis (o Redis con RediSearch) y un modelo de embeddings
4. Orquestador	Hermes Agent en Cloud Run	Hermes Agent en Azure Container Apps con ingreso interno	Mismos topes de pasos, timeout y tokens de salida
4. RAG	Cloud Storage + Vertex AI Vector Search	Blob Storage + Azure AI Search	Máximo 5 fragmentos

Capa	Google (este ADR)	Azure	Control en Azure
5. Egreso a LLMs	Apigee + Vertex AI	Backends de APIM + Microsoft Foundry	Balanceo (round-robin, ponderado, por prioridad) y circuit breaker según `Retry-After`. En Foundry: OpenAI, Claude (GA desde julio 2026) y modelos abiertos (Meta, Mistral, DeepSeek). APIM también enruta a Vertex AI o Bedrock
5. Egreso a tools	Agent Gateway + Agent Registry	APIM como gateway de MCP	Expone y gobierna servidores MCP; OAuth con su credential manager
5. Red privada	VPC + VPC Service Controls, Secret Manager	VNet + Private Endpoints, Key Vault	Sin IP pública; keys fuera del código
6. Detección y costo	Billing budgets, Security Command Center, Google SecOps	Cost Management (presupuestos), Azure Monitor con `llm-emit-token-metric`, Defender for Cloud para IA, Microsoft Sentinel	Métricas de tokens por usuario; alertas de prompt injection y exfiltración que llegan a Sentinel

Dónde se diferencia de Google:

- **Costo para el equipo:** Azure for Students da USD 100 de crédito por 12 meses sin tarjeta de crédito. Ojo: ese crédito no cubre Claude ni nada de Marketplace (ver la sección de viabilidad).
- **Borde más débil para T01/T04:** el rate limit de Front Door agrupa por IP de socket y sus contadores no siempre se comparten entre servidores del borde, así que con umbrales bajos se cuelan algunas peticiones. Cloud Armor permite agrupar también por huella TLS (JA4) o ASN.
- **Filtro de prompts:** Model Armor trae detección de datos sensibles y URLs maliciosas en el mismo filtro; la política `llm-content-safety` cubre categorías de daño, ataques al prompt y blocklists.
- **Caché semántica:** en Azure hay que desplegar y pagar Azure Managed Redis aparte.

Veredicto: Azure queda como alternativa equivalente en controles. Con presupuesto cero, Azure for Students solo sirve con Azure OpenAI como modelo; los créditos de docencia de Google son la otra vía sin tarjeta.

Consecuencias y validación

La arquitectura se valida con las mismas pruebas del repo: cada prueba debe mostrar que el ataque se frena en una capa concreta y antes del LLM.

Prueba	Ataque	Capa que debe frenarlo	Métrica a comparar Unsecure vs Secure
T01	Flooding de un usuario	Cloud Armor + `SpikeArrest` + `LLMTokenQuota`	Tokens gastados por el atacante, % de 429
T02	Pregunta que fuerza contexto masivo	`PromptTokenLimit` + tope de 5 fragmentos	`tokens_entrada` por petición
T03	Repetir la misma pregunta	Caché semántica	Tasa de aciertos de caché, latencia
T04	Ráfaga de muchos usuarios	Cloud Armor (JA4, ASN), reCAPTCHA, presupuesto global	Costo total estimado, peticiones que llegan al LLM
T05	Usuario legítimo durante el ataque	Cuotas por sesión e IP aíslan al atacante	Latencia p95 y % de éxito del usuario legítimo

Lo que ganamos: los controles ya no comparten proceso con la API atacada, la identidad no se puede falsificar desde el body y el gasto se corta en el borde.

Lo que cuesta: más servicios que configurar y pagar; hay que revisar si Apigee ofrece una organización de evaluación antes de desplegar.

Riesgos residuales:

- Sin login, un atacante puede abrir muchas sesiones desde IPs distintas (proxies) y cada una gasta dentro de su cuota. Lo frena el tope global y la huella TLS, no la cuota por sesión. El costo: si el atacante agota el tope global, el chatbot se cae para todos; se cambia denial of wallet por una denegación de servicio acotada y barata.
- La caché semántica puede devolver una respuesta de otro usuario si la pregunta tenía datos personales. Solo se cachean preguntas sobre material del curso.
- La estimación de tokens antes de llamar al LLM es aproximada; el descuento real se hace con la respuesta.

- Hermes Agent es open source y tiene acceso a herramientas; corre con cuenta de servicio de mínimo privilegio y solo con tools registradas.

Pregunta abierta: ¿la demo final corre en Google Cloud, en Azure (con el crédito de Azure for Students) o se simula en laptop con un gateway open source? Define qué columna de la tabla de componentes se implementa.

Viabilidad con créditos de estudiante

Con créditos de estudiante se puede desplegar casi toda la arquitectura, pero no los modelos de terceros: Claude (y en Google cualquier modelo de un socio) no se paga con crédito, solo con tarjeta. La demo con créditos queda con un proveedor propio de cada nube: Gemini en Google o Azure OpenAI en Azure.

Programa	Crédito	¿Tarjeta?	Qué bloquea para esta arquitectura
Google Cloud, créditos de docencia	Lo pide el profesor por curso; Colombia es elegible; vale 12 meses desde el inicio del curso	No	Según Google sirven para todos los servicios; la documentación no aclara si cubren Marketplace o modelos de socios (Claude en Vertex). Hay que confirmarlo con CloudEduGrants@google.com
Google Cloud, prueba gratuita	USD 300 por 90 días	Sí	Sin Marketplace, sin modelos generativos de terceros (Claude), sin aumento de cuotas
Azure for Students	USD 100 por 12 meses, renovable mientras sea estudiante	No	El crédito no paga productos de Marketplace ni de terceros. Claude en Foundry excluye cuentas de estudiante. Solo deja desplegar en unas cinco regiones por suscripción y a veces Foundry falla (caso de junio 2026). Al acabarse el crédito la suscripción se deshabilita

Otras restricciones que afectan los componentes:

- **Apigee:** tiene una organización de evaluación de 60 días gratis; el plan pago arranca cerca de USD 365 al mes por región según un comparador externo. Hay que verificar el precio oficial antes de presupuestar.
- **Azure API Management:** `llm-token-limit` no aparece para el nivel Consumption; se necesita al menos el nivel Developer, que consume crédito. El soporte para la API de Anthropic solo está en los niveles v2.

- **OpenAI directo:** requiere una API key con pago aparte; ningún crédito de estudiante la cubre.

Recomendación: pedirle al profesor los créditos de docencia de Google (sin tarjeta, encaja con lo del tablero), usar la organización de evaluación de Apigee durante la demo y dejar Gemini como modelo principal. Los otros proveedores quedan en el diseño como rutas de failover, documentadas pero no desplegadas.

Referencias

- OWASP LLM10:2025 Unbounded Consumption
- OWASP API4:2023 Unrestricted Resource Consumption
- Hands-On Architects: Denial of Wallet, cost-aware rate limiting
- Apigee: capacidades de AI gateway
- Apigee: política PromptTokenLimit
- Apigee: política LLMTokenQuota
- Apigee: política SemanticCacheLookup
- Model Armor: visión general
- Model Armor: integración con Apigee
- Cloud Armor: rate limiting
- Agent Gateway en Gemini Enterprise Agent Platform (codelab)
- Anuncios de seguridad de Google Cloud Next 2026
- Hermes Agent (Nous Research)
- Repo FDSI-GP-05

Azure:

- Azure API Management: AI gateway
- Azure API Management: política llm-content-safety
- Azure Front Door WAF: rate limiting
- Claude GA en Microsoft Foundry (Redmond Magazine)
- Zero Trust Assessment: Defender for Cloud para cargas de IA
- Azure for Students